

TYPE CASTING

class: 11

25/04/2025

1. Implicit Type Casting

2. Explicit Type Casting.

* Converting one datatype variable into another datatype - Type Casting

1. Implicit type Casting:

→ This is automatically done by Java.

Ex: byte b = 10;

short s = b; ✓

int i = b; ✓ (8) int i = s; ✓

long l = i; ✓

2. Explicit type Casting:

→ This can be done manually.

Ex: ① long l = 1000000000L;

int i = l; X $\leftarrow (-128 + (130 - 127))$

② short s = 128 $\leftarrow [-128 + (n-1)]$

byte b = s; X → Can write into ⇒ byte b = (byte) s; ✓

→ we have dataloss here. [∵ If we give out of range values its value decreases to negative].

Ex: Implicit type Casting:

package com;

public class Implicit {

public static void main (String[] args) {

byte b = 127;

short s = b;

s = 12000 int i = s;
System.out.println ("byte : " + b);

System.out.println ("short : " + s);

System.out.println ("int : " + i);

}

Output: byte : 127
short : 127.

Output: byte : 127
short : 12000
int : 12000

explicit type casting:

package com;

public class Explicit {

public static void main (String[] args) {

$383 - \overset{\text{input}}{256} - \text{short } s = \overset{128}{100};$

$127 - 0 - \underset{\text{output}}{\text{byte } b} = (\text{byte}) s;$

System.out.println ("short: " + s);

System.out.println ("byte: " + b);

output: short: 100
byte: 100.

short: 128

byte: -128

* Character:

short s = 97

char c = (char) s;

System.out.println ("short: " + s)

System.out.println ("char: " + c)

output: short: 97
char: a

* ASCII values [every character has definite values]

65 : A

int num = 'A'; → output: 65

System.out.println (num);

Ex1

package com;

public class NumberofDigits {

psum {

Scanner sc = new Scanner (System.in);

System.out.println ("Enter a num: ");

output: 15342

int num = sc.nextInt();

Num of digits: 5

int count = 0;

while (num > 0) {

num = num / 10;

count++;

}

System.out.println ("Num of digits: " + count);

}

② package com;

public class sumOf Digits {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

int sum = 0;

while (num > 0) {

sum = sum + num % 10;

num = num / 10;

System.out.println("sum of Digits: " + sum);

Output:

Enter a num:

153

sum of Digits: 9

STRING

- * String: Stores group of characters. Stores in double quotes (" ").
- * Character: Stores single character. Stores in single quotes (' ').

- * String:
 - is a non-primitive datatype.
 - object: String s = new String("New String"); → object.
 - String s = "New String"; → literals.

- * String Constant Pool: It is the special area in heap memory area to store creating string with literals.

- * In java, all strings are immutable.

- * Immutable: which doesn't change (final) i.e., we can't update.

- * If we create a string using literals, stored inside the String Constant Pools. Ex: String s = "FLM". we can update this string type. [i.e., create new string]

Ex: String s = "FLM";

s = "FLM Edutech"; → creates new string in memory.

s = "FLM Hyderabad"; → "

System.out.println(s); Output: FLM Hyderabad.

- * [String Constant Pools not create same strings, it will search for strings whether is there or not] & refers to that string] → If there is a same string, it will not create string again.
 - If there is no string, it creates new string.

- * String with object: stores in heap area.

→ ~~was~~ When comparing objects, it will compare address.

* == always compares address.

* $S_1 == S_2$ compares address

* $S_1.equals(S_2)$; compares content

* Hash code: like a address, has unique value.

$S_1 = "FLM"; \rightarrow \text{HashCode} : 69703$

$S_2 = "flm"; \rightarrow$

$sysout(S_1.hashCode()); \rightarrow 69703$

$sysout(S_2.hashCode()); \rightarrow 69703$

28/04/2025

* If we want to access methods & variables, we need to call that methods with object.

Ex: package com;
MethodsIn
public class String {
①
 PSVM {
 String s = "FLMedutech";
 String s1 = "FLMedutech";
 String st = new String("FLM")
 String st2 = new String("FLM")
 MethodsIn String obj = new MethodInString();
 obj.findlength();
 }
}

void findlength() {

output: 3

String s = "FLM"

sysout(s.length());

}

(8)

MethodInString obj = new MethodInString();

obj.findlength("FLM")

obj.findlength("FLMedutech")

output: 3
10

void findlength(Strings) {

sysout(s.length());

}

② Index: Starts with zero (0).

Method In String obj = new MethodInString();

obj.printCharAtIndex("FLM", 1);

obj.printCharAtIndex("FLMedutech" + 5)

```
void printCharAtIndex(String s, {  
    int index) {  
        s = "FLM"  
        syso(s.charAt(1));  
    }  
}
```

output:
FL
u

* If we not reach index, it will give out of bound exception.

③ Method In String obj = new MethodInString();

obj.checkIfEmpty("Hi");

obj.checkIfEmpty(" ");

```
void checkIfEmpty(String s) {
```

```
    syso(s.isEmpty()); → output: boolean values.  
    false.
```

④ Method In String obj = new MethodInString();

obj.checkIfBlank(" ");

obj.checkIfBlank(" ");

[Blank: It doesn't
care spaces]

```
void checkIfBlank(String s) {
```

```
    syso(s.isBlank());
```

output: True
False.

5

Method `String s1 = new String("FLM");`

`Obj: CheckIgnoreCase("FLM", "flm");`

`void CheckEqualsIgnoreCase(String s1, String s2) {`

`return s1.equalsIgnoreCase(s2); [Case insensitive]`

→ It will compare only content. Output: True.

6

`String s1 = "FLM"`

`String s2 = "FLM Edutech"`

Output: True.

`System.out.println(s2.contains(s1));`

`System.out.println(s2.startsWith("FLM"));` → Output: True.

~~and~~ `System.out.println(s2.endsWith("Edutech"));` → Output: True.

`System.out.println(s2.indexOf('L'));` → Output: 1

`System.out.println(s2.indexOf('e'));` → Output: 4

`System.out.println(s2.lastIndexOf('e'));` → Output: 8

`System.out.println(s2);`

*

7

SUBSTRING

→ If we want to get parts of string, we use substring.

`String s3 = "Programming";`

`System.out.println(s3.substring(0, 3));` → Output: Pro

[Need to take +1 extra]

`System.out.println(s3.substring(1, 4));` → Output: rog

`System.out.println(s3.substring(7, 11));` → Output: ming

`System.out.println(s3.substring(1));` → Output: rogramming.

`System.out.println(s3.substring(1, 12));` → Output: out of bound exception.

Trim:

* String s = " Hi " ;

syso(s.length()); → output: 7 (including spaces).

syso(s.trim()); → output: Hi

↓
removes empty spaces (i.e, white spaces);

syso(s.trim().length()); → s = s.trim();

syso(s.length()); → output: 3

syso(s.trim().length()); → output: 3

syso(s.length()); → output: 7

* "trim" is a temporary function. We have to assign that value like, s = s.trim();

* String s = " Hi FLM " ;

syso(s.length()); → output: 15

s = s.trim();

syso(s); → output: Hi FLM

syso(s.length()); → output: 10.

Replace: we can replace single & multiple characters at a time.

* String s₁ = " FLM Hyderabad " ;

syso(s₁.replace("H", "B")); → output: FLM Byderabad

syso(s₁.replace("Hyderabad", "Bangalore"));

output: FLM Bangalore.

→ This is also a temporary function.

String s₂ = "abcaabcaa";

syso(s₂.replace("aa", "bb"));

syso(s₂.replaceFirst("aa", "bb"));

Output: abcbbbcbb
abcbbbcbaa.

~~5450~~

String s₃ = "hello";

syso(s₃. toUpperCase()); → output: HELLO

syso(s₃. toLowerCase()); → output: hello.

Example: ~~5450~~

①

package com;

public class ReverseString {

public static void main (String[] args) {

ReverseString obj = new ReverseString();

String result = obj.reverse("FLM");

syso(result);

}

~~String~~ void

reverse (String s) {

String result = "";

for (int i = s.length() - 1; i >= 0; i--) {

(int i = s.length() - 1; i >= 0; i--)

Char temp = s.charAt(i);

result += temp;

}

return result;

output: MFL

}

}

(Never put return statement

inside the loop, because it will break the loop)

* The time complexity of String.concat() method is O(n), since it creates new string by copying both strings.


```

package com;
public class subsets {
    public static void main (String[] args) {
        Print Subsets Obj = new Subsets ();
        printsubSets ("abc");
    }
    void printSubSets (String s) {
        for (int i=0; i <= s.length() - 1; i++) {
            for (int j = i+1; j <= s.length(); j++) {
                syso (s.substring (i, j));
            }
        }
    }
}

```

output:

```

a
ab
abc
b
bc
c

```

* The String class in Java is immutable, meaning once created, its value cannot be changed. Any modifications result in a new String object.

* In String, The **equals()** method checks if two strings have the same content, while **==** checks if they reference the same object.

* In strings, the **join()** method joins multiple strings using a specified delimiter. ex: String.join("-", "A", "B") results in "A-B".

* **String.valueOf()** converts any primitive datatype & object into a String. ex: String.valueOf(1, 2, 3), returns "123".

* Java Strings immutable:

- to optimize usage (via pooling)
- to enhance security (since, data can't be altered)
- to enable String Pooling.
- supports hashCode caching.